

**DEPARTMENT OF INFORMATION TECHNOLOGY
JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM**



DJANGO FRAMEWORK LAB

R.BHANU VIGNESH (23VV1A1244)

SHABIRA BEGUM (23VV1A1246)

T.VISHAL (24VV5A1252)

Y.TEJASWINI (23VV1A1264)

UNDER GUIDANCE OF
MRS.MADHUMITA CHANDA

**DEPARTMENT OF INFORMATION
TECHNOLOGY**



**JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM**

Regd.No : 23VV1A1244

CERTIFICATE

This is to certify that this is a bonafide record of practical work done by MR. R.BHANU VIGNESH of IInd B.Tech IInd Semester Class in Django Framework Lab during the year 2024-25.

No.of Tasks Completed and Certified:12

Lecture In-Charge

Head of The Department

Date:



DEPARTMENT OF INFORMATION TECHNOLOGY

JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM

Website: www.jntugvcev.edu.in

Subject Name: DJANGO FRAMEWORK

Subject Code: R232212SE01

Year: 2025

Regulation: R23

COURSE OUTCOMES

NBA Subject Code	Course Outcomes	
	CO1	Design and build static as well as dynamic web pages and interactive web-based applications .
	CO2	Web development using Django framework.
	CO3	Analyze and create functional website in Django and deploy Django Web Application on Cloud .

CO-PO Mapping

Mapping of Course Outcomes (COs) with Program Outcomes (POs)

Course Outcomes		Program Outcomes (POs)														
		PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
	CO1	3	1	3	1	3	1	1	1	2	3	2	1	3	3	2
	CO2	3	2	3	1	3	1	1	1	2	2	2	2	3	3	3
	CO3	2	3	3	3	3	2	2	2	2	3	3	3	3	3	3

Enter correlation levels 1,2 and 3 as defined below:

1: Slight (Low) 2: Moderate (Medium) 3: Substantial (High) If there is no correlation, put “-”

Signature of the Course Instructor

Table of contents:

SNO	DATE	CONTENT	PAGE	MARKS	REMARKS
1	13-12-2024	Understanding Django and its Libraries	1-13		
2	20-12-2024	Introduction to Django Framework	14-15		
3	27-12-2024	Step-by-Step Guide to Installation of Django	16-19		
4	03-01-2025	Linking Views and URL Configurations	20-21		
5	24-01-2025	Exploring Django Views	22-26		
6	24-01-2025	Setting Up App-level URLs	27-29		
7	31-01-2025	Working with Templates in Django	30-58		
8	21-02-2025	Database Integration and Configuration SQLite	59-61		
9	21-02-2025	Handling Forms in Django	62-65		
10	21-02-2025	Defining and Using Models	66-69		
11	07-03-2025	Migration Sync with the Database	70-71		
12	27-03-2025	Deploying Django Application on cloud platforms	72-73		
13	04-04-2025	Front End Web Developer Certification	74		

Date :

signature:



**DEPARTMENT OF INFORMATION TECHNOLOGY
JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM**

Dr.Ch. Bindu Madhuri
Asst. Professor & HOD

Email: hod. it@intugvcev.edu.in

1. Name of the Laboratory : Django Framework
2. Name of the Student : R.Bhanu Vignesh
3. Roll No : 23VV1A1244
4. Class : II B-Tech II Semester
5. Academic Year : 2024-25
6. Name of Experiment : Understanding Django and its libraries
7. Date of Experiment : 13-12-2024
8. Date of Submission of Report : 20-12-2024

S,NO	ABILITY AND ACTIVITY	WEIGHTAGE OF MARKS	DAY TO DAY EVALUATION SCORE
1	Aim Objective, Tools required	3	
2	Theory, Algorithm and Observations	3	
3	Implementation	3	
4	Schematic diagrams, Architecture, workflow, Flowchart	3	
5	Tidiness of his/her working area, proper maintenance of system during and after experiment.	3	
	Total Score	15	

DATE:

Signature of Faculty :

Understanding Django and Its Libraries

PYTHON LIBRARIES

1. Python Collections - Container Datatypes:

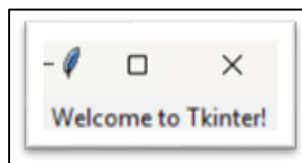
- a. **Purpose:** Provides specialized container datatypes that support efficient handling of data.
- b. **Key Types:**
 - i. **List:** Ordered, mutable, allows duplicates.
 - ii. **Tuple:** Ordered, immutable, allows duplicates.
 - iii. **Set:** Unordered, no duplicates, fast membership testing.
 - iv. **Dictionary:** Unordered, key-value pairs, fast lookups.
- c. **Common Use:** Data manipulation, storing and accessing collections of data in web apps (like user data or API responses).

2. Tkinter - GUI Applications:

- a. **Purpose:** Python's standard library for creating graphical user interfaces (GUIs).
- b. **Key Features:**
 - i. Widgets: Buttons, labels, text boxes, etc.
 - ii. Event handling: Respond to user interactions like clicks or key presses.
 - iii. Simple layout management.
- c. **Common Use:** Build desktop applications or tools for local interaction with a web app backend.

Code:

```
from tkinter import Tk, Label
# Create a window
root = Tk()
root.title("Hello Window")
# Add a label to display text
Label(root, text="Welcome to Tkinter!").pack()
# Run the application
root.mainloop()
```

Output:**3. Requests - HTTP Requests:**

- a. **Purpose:** Simplifies HTTP requests to interact with web APIs.
- b. **Key Features:**
 - i. Send GET, POST, PUT, DELETE requests easily.
 - ii. Handle request parameters, headers, and cookies.
 - iii. Simple error handling and response handling.
- c. **Common Use:** Interact with REST APIs, download content from the web.

Code:

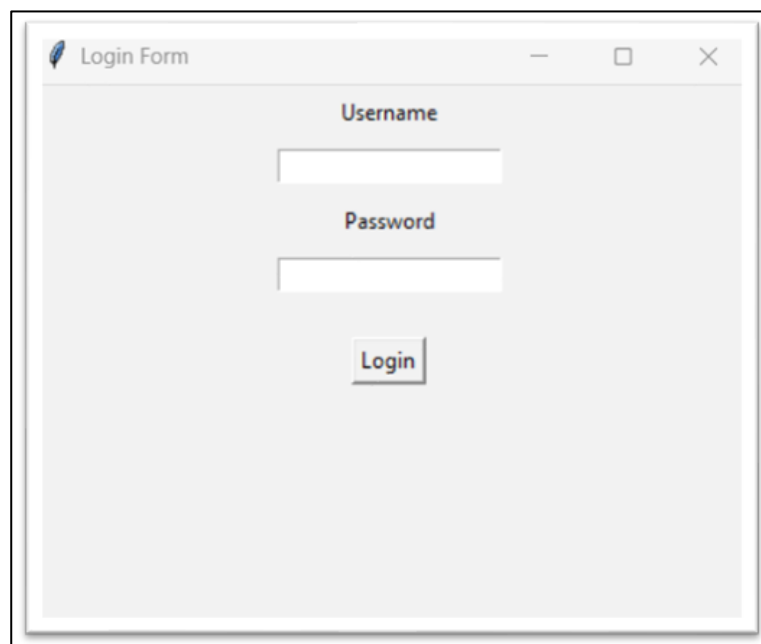
```
# Tkinter app (internet not needed; standard Python installations)
# pip install tk (usually not necessary for Tkinter, as it comes pre-installed)

import tkinter as tk
from tkinter import messagebox

def validate_login():
    username = username_entry.get()
    password = password_entry.get()
```

```
if username == "user" and password == "123":  
    messagebox.showinfo("Login Success", "Login successful")  
else:  
    messagebox.showerror("Login Failed", "Invalid username or password")  
  
# Run the Tkinter app  
root = tk.Tk()  
root.title("Login Form")  
root.geometry("300x150")  
  
# Username input  
username_label = tk.Label(root, text="Username")  
username_label.pack(pady=5)  
username_entry = tk.Entry(root)  
username_entry.pack(pady=5)  
  
# Password input  
password_label = tk.Label(root, text="Password")  
password_label.pack(pady=5)  
password_entry = tk.Entry(root, show="*")  
password_entry.pack(pady=5)  
  
# Login button  
login_button = tk.Button(root, text="Login", command=validate_login)  
login_button.pack(pady=10)  
  
root.mainloop()
```

Output:



4. Scrapy:

- a. **Purpose:** An open-source web crawling framework for large-scale web scraping.
- b. **Key Features:**
 - i. Fast, extensible, and asynchronous web scraping.
 - ii. Supports handling requests, data extraction, and storing results.
 - iii. Built-in handling for logging, retries, and sessions.
- c. **Common Use:** Web crawling and scraping projects that require high performance.

5. BeautifulSoup4 - Web Scraping:

- a. **Purpose:** Parses HTML and XML documents to extract data.
- b. **Key Features:**
 - i. Easy navigation and searching within HTML.
 - ii. Supports different parsers like html.parser, lxml, and html5lib.
- c. **Common Use:** Extract data from websites for analysis, e.g., for building data-driven applications.

Code:

```
import requests
from bs4 import BeautifulSoup
import re

# The URL of the website to scrape
url = 'https://example.com'

# Set headers to avoid getting blocked
headers = {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
    AppleWebKit/537.36 (KHTML, like Gecko) Chrome/100.0.4896.127
    Safari/537.36'
}

try:
    # Send a GET request
    response = requests.get(url, headers=headers, timeout=10)
```

```

response.raise_for_status()

# Parse the content
soup = BeautifulSoup(response.text, 'html.parser')

# Get the page title
title_tag = soup.find('title')
if title_tag:
    print(f"Page title: {title_tag.text.strip()}")
else:
    print("No title found")

# Extract and print all links
links = soup.find_all('a')
for link in links:
    href = link.get('href')
    if href:
        print(href)

except requests.exceptions.RequestException as e:
    print(f"Error fetching the webpage: {e}")

```

Output:

```

S C:\Users\bhanu\OneDrive\Desktop\python python
C:/Users/bhanu/OneDrive/Desktop/python/Basic/New folder/demo.py"
Title of page: Example Domain
Headings:
>> -h1: Example Domain
>>
>>Title of the page: Example Domain

```

6. Zappa:

- a. **Purpose:** Deploy Python web applications to AWS Lambda and API Gateway.
- b. **Key Features:**
 - i. Supports frameworks like Flask and Django for serverless deployments.
 - ii. Manages serverless architecture and deployment configurations.

- c. **Common Use:** Build scalable, serverless web apps without maintaining servers.

7. Dash:

- a. **Purpose:** Web application framework for building interactive data visualization applications.
- b. **Key Features:**
 - i. Built on top of Flask, React, and Plotly.
 - ii. Integrates seamlessly with data science libraries (e.g., Pandas, Plotly).
- c. **Common Use:** Building dashboards and data-driven web applications.

8. TurboGears:

- a. **Purpose:** Full-stack web framework built on top of WSGI.
- b. **Key Features:**
 - i. Modular: Mix and match components like SQLAlchemy, Genshi, and others.
 - ii. Focus on rapid development and scalability.
- c. **Common Use:** Develop scalable, enterprise-level web applications.

9. CherryPy:

- a. **Purpose:** Minimalistic web framework for building web applications.
- b. **Key Features:**
 - i. Provides a simple and fast HTTP server.
 - ii. Handles routing, cookies, sessions, and file uploads.
- c. **Common Use:** Building web applications with a lightweight framework.

Code:

```

import cherrypy

class HelloWorld:
    @cherrypy.expose
    def index(self):
        return "Hello, World! This is a CherryPy web page."

if __name__ == '__main__':
    # Server Configuration (Custom Port & Logging)
    cherrypy.config.update({
        'server.socket_port': 9090, # Change the default port 8080
        'log.error_file': 'cherrypy_error.log', # Log errors
        'log.access_file': 'cherrypy_access.log' # Log access
    })

    print("Server is running on http://127.0.0.1:9090\n")

    try:
        cherrypy.quickstart(HelloWorld())
    except Exception as e:
        print(f"Error starting CherryPy: {e}")

```

Output:

```

CherryPy is installed!

C:\Users\bhanu\OneDrive\Desktop\pythonC:\Users\bhanuOneDrive\Desktop\python
\venv\Scripts\python.exe

C:\Users\bhanu\OneDrive\Desktop\python\Basic\New folder\Books\cherry.py

Server is running on http://127.0.0.1:8080

[15/Nov/2025:21:24:13] ENGINE Started for console output.

[15/Nov/2025:21:24:13] ENGINE Started monitor thread 'Autoreloader'.

[15/Nov/2025:21:24:14] ENGINE Serving on http://127.0.0.1:8080

127.0.0.1 - - [15/Nov/2025:21:24:18] "GET /favicon.ico HTTP/1.1" 200 -

127.0.0.1 - - [15/Nov/2025:21:24:18] "GET /favicon.ico HTTP/1.1" 200 -

Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/110.0...

```

After run the server :-



10. Flask:

- a. **Purpose:** Lightweight micro-framework for building web applications.
- b. **Key Features:**
 - i. Simple to learn and use, but highly extensible.
 - ii. Supports extensions for database integration, form handling, authentication, etc.
- c. **Common Use:** Small to medium web applications, APIs, or microservices.

Code:

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    return 'Hello, World!'

if __name__ == '__main__':
    app.run(debug=True)
```

Output:

```

C:\Users\user\OneDrive\Desktop\python Basic\New folder\Books>python app.py
* Serving Flask app 'app'
Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
Running on http://127.0.0.1:5000
Press CTRL-C to quit
• Restarting with stat
Debugger is active
127/mar/2025 22:53:51] "GET/HPP 208
127/mar/2025 22:53:51) PRT/favicon.ico 0-
C:\Users\user\OneDrive\Desktop\python
Basic\New folder\Books >

```

After run the server:



Hello, World!

11. Web2Py:

- a. **Purpose:** Full-stack framework for rapid web application development.
- b. **Key Features:**
 - i. Includes a web-based IDE for development.
 - ii. Built-in ticketing system and database integration.
- c. **Common Use:** Enterprise web applications with minimal setup.

12.Bottle:

- a. **Purpose:** Simple and lightweight WSGI micro-framework.
- b. **Key Features:**
 - i. Single-file framework, minimalistic, and fast.
 - ii. No dependencies, supports routing, templates, and form handling.
- c. **Common Use:** Small web applications, APIs, and prototypes.

Code:

```
I Crem bottle import Bottle, run

# Create a Bottle application instance app = Bottle()

@app.route("")

def index()

return Hello. World! This is a Bottle web page.

# Run the Bottle application

if __name__ == '__main__':

run(app, host='localhost', port=8080,
```

Output:

```
C:\Users\bhanu\OneDrive\python Basic\New folder\Books>python bottle_app.
Bottle v0.13.2 server starting up (using WSGIefServer)...
Listening on http://localhost:8080/
Hit Ctrl-C to quit.
```

After run the server:



13.Falcon:

- a. **Purpose:** High-performance framework for building APIs.
- b. **Key Features:**
 - i. Focuses on speed and minimalism.
 - ii. Supports RESTful API development and is optimized for large-scale deployments.
- c. **Common Use:** Building fast, high-performance APIs.

14.CubicWeb:

- a. **Purpose:** Web application framework based on an entity-relation model.
- b. **Key Features:**
 - i. Uses a highly modular architecture for development.
 - ii. Focus on building web apps with rich data models.
- c. **Common Use:** Semantic web applications or data-driven web apps.

15.Quixote:

- a. **Purpose:** A web framework designed for simplicity and scalability.
- b. **Key Features:**
 - i. Full support for Python's object-oriented programming.
 - ii. Easily extensible, with minimalistic core.
- c. **Common Use:** Scalable and customizable web applications.

16.Pyramid:

- a. **Purpose:** Full-stack web framework that can scale from simple to complex applications.
- b. **Key Features:**
 - i. Highly flexible with support for routing, templating, authentication, and authorization.
 - ii. Allows for small and large applications, with fine-grain control.
- c. **Common Use:** Building large, enterprise-grade web applications and REST APIs.

SUMMARY:

1. **Flask, Django, Pyramid:** Popular web frameworks, each offering flexibility and scalability.
2. **Scrapy, BeautifulSoup4:** Specialized for web scraping and data extraction.
3. **Requests, Zappa, Dash:** Tools for making HTTP requests, serverless apps, and interactive data visualizations.
4. **Tkinter, Bottle, CherryPy:** Libraries for building lightweight desktop and web applications.



DEPARTMENT OF INFORMATION TECHNOLOGY
JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM

Dr.Ch. Bindu Madhuri
 Asst. Professor & HOD

Email: hod. it@intugvcev.edu.in

1. Name of the Laboratory : Django Framework
 2. Name of the Student : R.Bhanu Vignesh
 3. Roll No : 23VV1A1244
 4. Class : II B-Tech II Semester
 5. Academic Year : 2024-25
 6. Name of Experiment : Introduction to Django Framework
 7. Date of Experiment : 20-12-2024
 8. Date of Submission of Report : 27-12-2024

S,NO	ABILITY AND ACTIVITY	WEIGHTAGE OF MARKS	DAY TO DAY EVALUATION SCORE
1	Aim Objective, Tools required	3	
2	Theory, Algorithm and Observations	3	
3	Implementation	3	
4	Schematic diagrams, Architecture, workflow, Flowchart	3	
5	Tidiness of his/her working area, proper maintenance of system during and after experiment.	3	
	Total Score	15	

DATE:

Signature of Faculty :

Introduction to Django Framework

Django: A Web Framework for Python

- i. Django: Django is a **high-level Python web framework** that allows developers to build secure, scalable, and maintainable web applications **quickly and efficiently**. It follows the **Model-View-Template (MVT)** architectural pattern.

Key Features of Django:

1. Fast Development – Comes with built-in features like authentication, database management, and an admin panel.
2. Scalability – Suitable for small projects to enterprise-level applications.
3. Security – Protects against common security threats (SQL Injection, CSRF, XSS, etc.).
4. ORM (Object-Relational Mapper) – Allows database interaction using Python instead of SQL.
5. Built-in Admin Panel – Auto-generates an admin interface for managing data.
6. Reusable App-Developers can create modular and reusable components.

Django's MVT Architecture:

1. Model (M) – Handles database interactions (e.g., User, Booking).
2. View (V) – Manages business logic and connects models to templates.
3. Template (T) – Renders HTML pages dynamically.

Example MVT Folder Structure in Django

```

myproject/      # Project Directory
├── myproject/  # Project Settings Directory
│   ├── __init__.py
│   ├── settings.py # Project settings
│   ├── urls.py    # URL routing
│   ├── wsgi.py    # WSGI entry point
│   └── asgi.py    # ASGI entry point
├── myapp1/     # Django App Directory
│   ├── migrations/ # Database migrations
│   ├── __init__.py
│   ├── admin.py   # Admin panel configurations
│   ├── apps.py    # App configuration
│   ├── models.py  # Models (Database structure)
│   ├── views.py   # Business logic
│   ├── urls.py    # App-specific URLs
│   └── templates/ # Template folder
│       └── home.html # HTML file for rendering UI
└── manage.py     # Django command-line utility
  
```



**DEPARTMENT OF INFORMATION TECHNOLOGY
JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM**

Dr.Ch. Bindu Madhuri
Asst. Professor & HOD

Email: hod. it@intugvcev.edu.in

1. Name of the Laboratory : Django Framework
2. Name of the Student : R.Bhanu Vignesh
3. Roll No : 23VV1A1244
4. Class : II B-Tech II Semester
5. Academic Year : 2024-25
6. Name of Experiment : Step-by-Step Guide to Installing of Django
7. Date of Experiment : 27-12-2024
8. Date of Submission of Report : 03-01-2025

S,NO	ABILITY AND ACTIVITY	WEIGHTAGE OF MARKS	DAY TO DAY EVALUATION SCORE
1	Aim Objective, Tools required	3	
2	Theory, Algorithm and Observations	3	
3	Implementation	3	
4	Schematic diagrams, Architecture, workflow, Flowchart	3	
5	Tidiness of his/her working area, proper maintenance of system during and after experiment.	3	
	Total Score	15	

DATE:

Signature of Faculty :

Step-by-step Guide to Installing Django

Step-1 : Checking the installation & version of Python PIP

```
python --version
```

```
pip --version
```

Step-2 : Installation of Virtual Environment

```
pip install virtualenvwrapper-win
```

Step-3 : Creation of Virtual Environment

```
mkvirtualenv (name)
```

Step-4 : Installation of Django in Virtual environment

```
pip install Django
```

Step-5 : Create a folder to store all the projects

```
mkdir proj_folder_name
```

Step-6 : Start new_project

```
django-admin startproject project_name
```

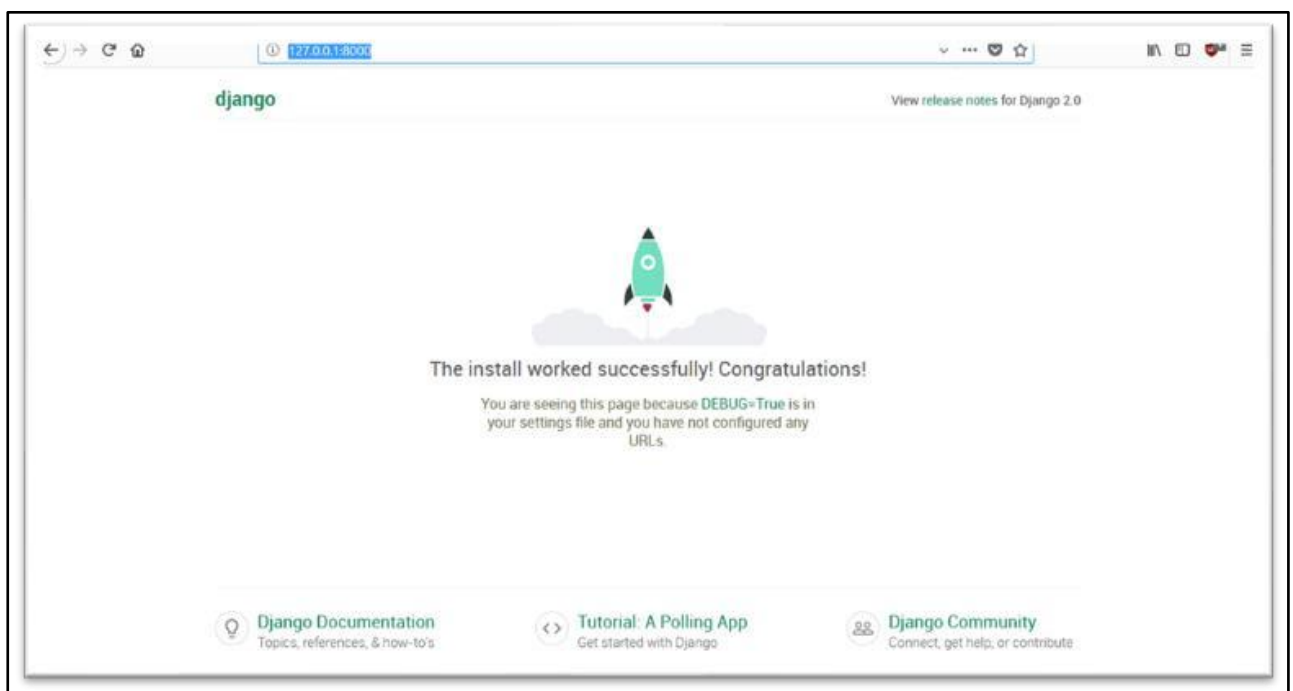
```
django-admin startapp app_name
```

Step-7 : Run the server

```
python manage.py runserver
```

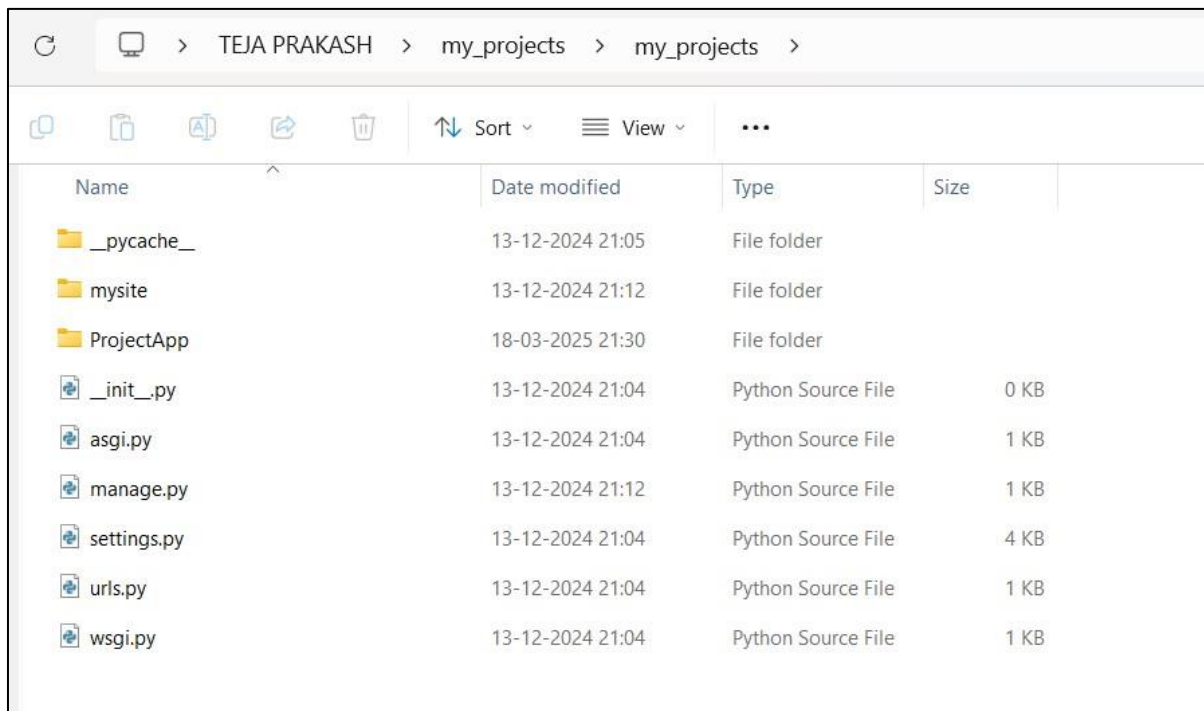
Step-8 : Open the browser and check the homepage of Django..

OUTPUT:



PROJECT CREATION

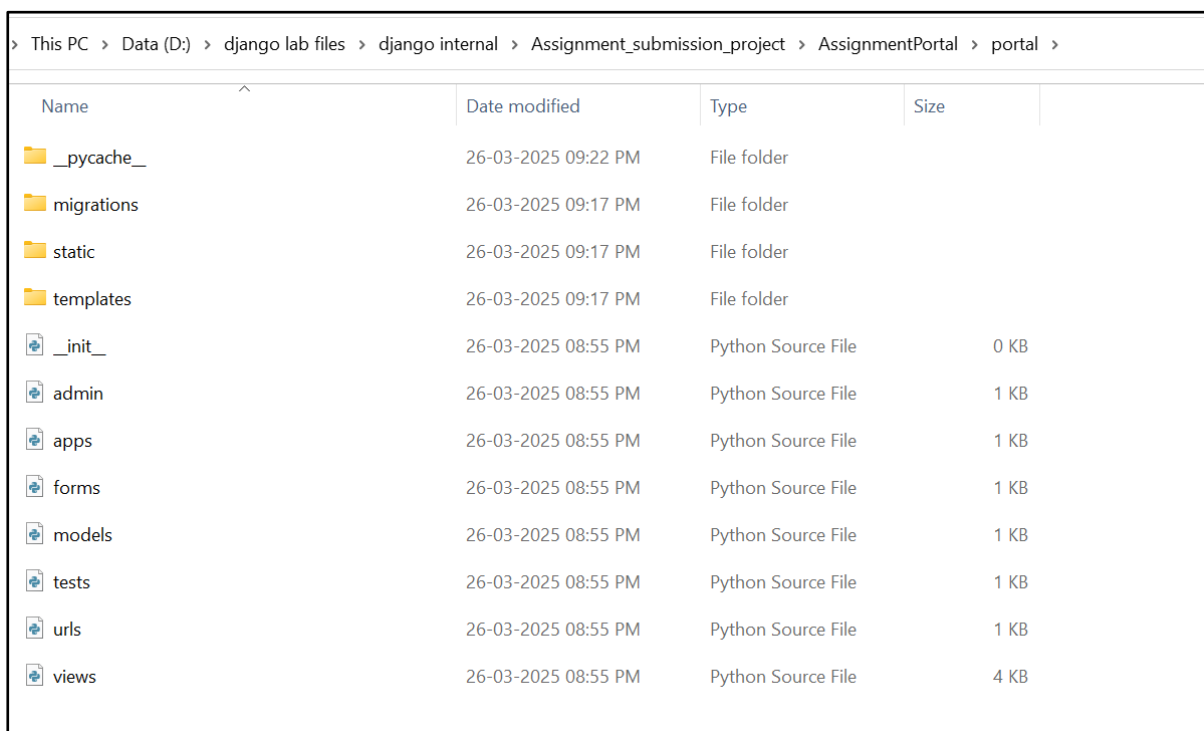
Command used : `django-admin startproject project_name`



Name	Date modified	Type	Size
__pycache__	13-12-2024 21:05	File folder	
mysite	13-12-2024 21:12	File folder	
ProjectApp	18-03-2025 21:30	File folder	
__init__.py	13-12-2024 21:04	Python Source File	0 KB
asgi.py	13-12-2024 21:04	Python Source File	1 KB
manage.py	13-12-2024 21:12	Python Source File	1 KB
settings.py	13-12-2024 21:04	Python Source File	4 KB
urls.py	13-12-2024 21:04	Python Source File	1 KB
wsgi.py	13-12-2024 21:04	Python Source File	1 KB

APP CREATION

Command used : `django-admin startproject app_name`



Name	Date modified	Type	Size
__pycache__	26-03-2025 09:22 PM	File folder	
migrations	26-03-2025 09:17 PM	File folder	
static	26-03-2025 09:17 PM	File folder	
templates	26-03-2025 09:17 PM	File folder	
__init__.py	26-03-2025 08:55 PM	Python Source File	0 KB
admin	26-03-2025 08:55 PM	Python Source File	1 KB
apps	26-03-2025 08:55 PM	Python Source File	1 KB
forms	26-03-2025 08:55 PM	Python Source File	1 KB
models	26-03-2025 08:55 PM	Python Source File	1 KB
tests	26-03-2025 08:55 PM	Python Source File	1 KB
urls	26-03-2025 08:55 PM	Python Source File	1 KB
views	26-03-2025 08:55 PM	Python Source File	4 KB



**DEPARTMENT OF INFORMATION TECHNOLOGY
JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM**

Dr.Ch. Bindu Madhuri
Asst. Professor & HOD

Email: hod. it@intugvcev.edu.in

1. Name of the Laboratory : Django Framework
2. Name of the Student : R.Bhanu Vignesh
3. Roll No : 23VV1A1244
4. Class : II B-Tech II Semester
5. Academic Year : 2024-25
6. Name of Experiment : Linking Views and URLConfigurations
7. Date of Experiment : 03-01-2025
8. Date of Submission of Report : 24-01-2025

S,NO	ABILITY AND ACTIVITY	WEIGHTAGE OF MARKS	DAY TO DAY EVALUATION SCORE
1	Aim Objective, Tools required	3	
2	Theory, Algorithm and Observations	3	
3	Implementation	3	
4	Schematic diagrams, Architecture, workflow, Flowchart	3	
5	Tidiness of his/her working area, proper maintenance of system during and after experiment.	3	
	Total Score	15	

DATE:

Signature of Faculty :

Linking Views and URL Configurations

Set Up URLs:

Project-level URL Configuration:

```
from django.contrib import admin
from django.urls import path, include
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('myapp1.urls')), # Include app URLs
]
```

App-level URL Configuration

```
from django.urls import path
from .views import home
urlpatterns = [
    path('', home, name='home'),
]
```

Create a Sample View:

```
from django.http import HttpResponse
def home(request):
    return HttpResponse("<h1>Welcome to My Django App!</h1>")
```

Run Migrations:

```
python manage.py migrate
```

Run the Server and Test:

```
python manage.py runserver
```



DEPARTMENT OF INFORMATION TECHNOLOGY
JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM

Dr.Ch. Bindu Madhuri
 Asst. Professor & HOD

Email: hod. it@intugvcev.edu.in

1. Name of the Laboratory : Django Framework
2. Name of the Student : R.Bhanu Vignesh
3. Roll No : 23VV1A1244
4. Class : II B-Tech II Semester
5. Academic Year : 2024-25
6. Name of Experiment : Exploring Django Views
7. Date of Experiment : 24-01-2025
8. Date of Submission of Report : 24-01-2025

S,NO	ABILITY AND ACTIVITY	WEIGHTAGE OF MARKS	DAY TO DAY EVALUTION SCORE
1	Aim Objective, Tools required	3	
2	Theory, Algorithm and Observations	3	
3	Implementation	3	
4	Schematic diagrams, Architecture, workflow, Flowchart	3	
5	Tidiness of his/her working area, proper maintenance of system during and after experiment.	3	
	Total Score	15	

DATE:

Signature of Faculty :

Exploring Django Views

The views.py file in Django is where you define the logic for handling user requests and generating responses. It's a key part of the MVT (Model-View-Template) architecture, acting as the **"View" layer**. It contains functions or classes (called views) that process incoming web requests, like a user visiting a page or submitting a form.

```
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.forms import AuthenticationForm
from django.contrib.auth.decorators import login_required
from django.utils.timezone import now
from .models import Assignment, Submission, UserProfile
from .forms import SignupForm, SubmissionForm
from django.db import IntegrityError

def user_signup(request):
    if request.method == "POST":
        form = SignupForm(request.POST)
        if form.is_valid():
            user = form.save(commit=False)
            user.set_password(form.cleaned_data['password1']) # Hash password
            user.save()

            role = form.cleaned_data.get('role', 'student') # Default role is student

            # Check if UserProfile already exists before creating
            user_profile, created = UserProfile.objects.get_or_create(user=user,
defaults={'role': role})

            if not created:
                # If the profile already exists, update the role if needed
                user_profile.role = role
                user_profile.save()

            login(request, user)

            # Redirect based on role after signup
            if role == "admin":
                return redirect('admin_dashboard')
            elif role == "teacher":
                return redirect('view_submissions')
            return redirect('assignment_list')

        else:
            form = SignupForm()

    return render(request, 'signup.html', {'form': form})

# FIXED Signup Function (Admins Redirect to Admin Dashboard)
```

```

# in Function (Avoided Duplicate Definition)
def user_login(request):
    if request.method == "POST":
        form = AuthenticationForm(data=request.POST)
        if form.is_valid():
            user = form.get_user()
            login(request, user)

            # Redirect based on role
            profile = UserProfile.objects.filter(user=user).first()
            if profile and profile.role == "admin":
                return redirect('admin_dashboard')
            elif profile and profile.role == "teacher":
                return redirect('view_submissions')
            return redirect('assignment_list')
        else:
            form = AuthenticationForm()
            return render(request, 'login.html', {'form': form})

def user_logout(request):
    logout(request)
    return redirect('user_login')

@login_required
def dashboard(request):
    # Ensure user has a UserProfile, create if missing
    profile, created = UserProfile.objects.get_or_create(user=request.user,
defaults={'role': 'student'})

    if profile.role == 'student':
        return redirect('assignment_list')
    elif profile.role == 'teacher':
        return redirect('view_submissions')
    elif profile.role == 'admin':
        return redirect('admin_dashboard')

    return redirect('user_login') # Fallback if role is not recognized

@login_required
def assignment_list(request):
    assignments = Assignment.objects.all()
    return render(request, 'assignment_list.html', {'assignments': assignments})

```

code continues...

```

@login_required
def submit_assignment(request, assignment_id):
    assignment = get_object_or_404(Assignment, id=assignment_id)
    if request.method == "POST":
        form = SubmissionForm(request.POST, request.FILES)
        if form.is_valid():
            submission = form.save(commit=False)
            submission.student = request.user
            submission.assignment = assignment
            submission.save()
            return redirect('assignment_list')
    else:
        form = SubmissionForm()
    return render(request, 'submit_assignment.html', {'form': form, 'assignment':
assignment})

@login_required
def view_submissions(request):
    profile, created = UserProfile.objects.get_or_create(user=request.user,
defaults={'role': 'student'})
    if profile.role != 'teacher':
        return redirect('dashboard')
    submissions = Submission.objects.all()
    return render(request, 'view_submissions.html', {'submissions': submissions})

@login_required
def grade_submission(request, submission_id):
    profile, created = UserProfile.objects.get_or_create(user=request.user,
defaults={'role': 'student'})
    if profile.role != 'teacher':
        return redirect('dashboard')

    submission = get_object_or_404(Submission, id=submission_id)
    if request.method == "POST":
        submission.grade = request.POST['grade']
        submission.feedback = request.POST['feedback']
        submission.save()
        return redirect('view_submissions')
    return render(request, 'grade_submission.html', {'submission': submission})

@login_required
def admin_dashboard(request):
    profile, created = UserProfile.objects.get_or_create(user=request.user,
defaults={'role': 'student'})

    if profile.role != 'admin': # Ensure only admins can access
        return render(request, '403.html')

```

```
total_users = UserProfile.objects.count()
total_assignments = Assignment.objects.count()
total_submissions = Submission.objects.count()
pending_submissions = Submission.objects.filter(grade__isnull=True).count()

# Fetch upcoming deadlines
upcoming_assignments =
Assignment.objects.filter(deadline__gte=now()).order_by('deadline')

context = {
    'total_users': total_users,
    'total_assignments': total_assignments,
    'total_submissions': total_submissions,
    'pending_submissions': pending_submissions,
    'upcoming_assignments': upcoming_assignments,
}

return render(request, 'admin_dashboard.html', context)
```



DEPARTMENT OF INFORMATION TECHNOLOGY
JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM

Dr.Ch. Bindu Madhuri
 Asst. Professor & HOD

Email: hod. it@intugvcev.edu.in

1. Name of the Laboratory : Django Framework
2. Name of the Student : R.Bhanu Vignesh
3. Roll No : 23VV1A1244
4. Class : II B-Tech II Semester
5. Academic Year : 2024-25
6. Name of Experiment : Setting Up App-Level URL's
7. Date of Experiment : 24-01-2025
8. Date of Submission of Report : 31-01-2025

S,NO	ABILITY AND ACTIVITY	WEIGHTAGE OF MARKS	DAY TO DAY EVALUTION SCORE
1	Aim Objective, Tools required	3	
2	Theory, Algorithm and Observations	3	
3	Implementation	3	
4	Schematic diagrams, Architecture, workflow, Flowchart	3	
5	Tidiness of his/her working area, proper maintenance of system during and after experiment.	3	
	Total Score	15	

DATE:

Signature of Faculty :

Setting Up App-Level URLs

Creating urls.py in a Django App:

In Django, urls.py is a configuration file used to define URL patterns and map them to specific views in your application. It acts as a routing system, directing incoming HTTP requests to the appropriate view functions or classes based on the URL.

Steps for creating urls.py:

Locate or Create the File:

- i. For the project: Use the existing urls.py in the project directory.
- ii. For an app: Create a new urls.py file inside your app folder (e.g., myapp/).

Add Basic Structure:

- i. Import path from django.urls.
- ii. Import your app's views module.
- iii. Create a list named urlpatterns.

Define URL Patterns:

- i. Use path(route, view, name) to add URL mappings to urlpatterns.

Save the File.

App Urls.Py :

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.login_view, name='login'),
    path('logout/', views.logout_view, name='logout'),
    path('student/', views.student_dashboard, name='student_dashboard'),
    path('teacher/', views.teacher_dashboard, name='teacher_dashboard'),
    path('assignment/create/', views.create_assignment, name='create_assignment'),
    path('assignment/<int:assignment_id>/submit/', views.submit_assignment,
name='submit_assignment'),
    path('assignment/<int:assignment_id>/submissions/', views.view_submissions,
name='view_submissions'),
]
```


Connecting App urls.py to Project urls.py:

To connect an app's urls.py to the project's urls.py in Django, you use the **include()** function in the project's root urls.py. This links the app's URL patterns to a specific base URL, enabling modular routing.

- i. **Ensure App urls.py Exists:** Create and define URL patterns in your app's urls.py (e.g., myapp/urls.py).
- ii. **Edit Project urls.py:** In the project's urls.py (e.g., myproject/urls.py), import include from django.urls.
- iii. **Add Path with include():** Use path('base_url/', include('app_name.urls')) in the urlpatterns list to link the app's URLs.
- iv. **Test the Connection:** Run the server and access the app's URLs prefixed with the base URL.

Project Urls.Py :

To use the app's URLs, include them in the project-level urls.py (AssignmentPortal/urls.py).

```
from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path("", include('portal.urls')),
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```



DEPARTMENT OF INFORMATION TECHNOLOGY
JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM

Dr.Ch. Bindu Madhuri
 Asst. Professor & HOD

Email: hod. it@intugvcev.edu.in

1. Name of the Laboratory : Django Framework
2. Name of the Student : R.Bhanu Vignesh
3. Roll No : 23VV1A1244
4. Class : II B-Tech II Semester
5. Academic Year : 2024-25
6. Name of Experiment : Working with Templates in Django
7. Date of Experiment : 31-01-2025
8. Date of Submission of Report : 17-02-2025

S,NO	ABILITY AND ACTIVITY	WEIGHTAGE OF MARKS	DAY TO DAY EVALUTION SCORE
1	Aim Objective, Tools required	3	
2	Theory, Algorithm and Observations	3	
3	Implementation	3	
4	Schematic diagrams, Architecture, workflow, Flowchart	3	
5	Tidiness of his/her working area, proper maintenance of system during and after experiment.	3	
	Total Score	15	

DATE:

Signature of Faculty :

Working with Templates in Django

In a Django project, the **templates folder** is used to store HTML files (templates) that define the structure and presentation of your web pages. Django's template engine renders these files dynamically by combining them with data from your views.

Key points about templates:

Location

1. The templates folder is typically placed inside each app (e.g., myapp/templates/) or at the project level (e.g., myproject/templates/) for shared templates.

Configuration

1. In settings.py, the TEMPLATES setting with 'APP_DIRS': True enables Django to find app-level templates, or you can add custom paths to DIRS.

Naming Convention

1. Templates are often stored in app-named subfolders (e.g., myapp/templates/myapp/) to prevent naming conflicts across apps.

Usage

1. Templates use Django's template language and are rendered by views with the render() function to display dynamic content.

Where to Store Templates?

App-Level: Inside each app, create a templates folder (e.g., portal/templates/). For organization, use a subfolder named after the app (e.g., portal/templates/portal/home.html) to avoid conflicts.

Project-Level: For shared templates, create a templates folder in the project root (e.g., AssignmentPortal/templates/). Update settings.py to include this path in TEMPLATES['DIRS'].

```

assignmentportal/
├── manage.py
├── db.sqlite3
├── assignmentportal/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
├── portal/
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── migrations/
│   │   └── __init__.py
│   ├── models.py
│   ├── forms.py
│   ├── views.py
│   ├── urls.py
│   ├── static/
│   │   ├── css/
│   │   │   └── style.css
│   │   └── js/
│   │       └── script.js
│   └── templates/
│       └── portal/
│           ├── base.html
│           ├── login.html
│           ├── assignmentsubmission.html
│           ├── assignmentcreate.html
│           ├── submissionview.html
│           ├── student_board.html
│           └── teacher_board.html
└── requirements.txt

```

base.html :

```

{% load static %}
<!DOCTYPE html>
<html>
<head>
<title>Assignment Portal</title>
<link rel="stylesheet" href="{% static 'css/style.css' %}">
<style>
  /* General body styling */
  body {
    margin: 0;
    font-family: Arial, sans-serif;
    background-color: #f0f4f8; /* Light blue-gray background */
  }

  /* Navbar styling */
  .navbar {
    background-color: #2c3e50; /* Dark blue */
    padding: 15px 20px;
    display: flex;
    justify-content: flex-end;
    box-shadow: 0 2px 5px rgba(0, 0, 0, 0.2);
  }

  .navbar a {
    color: #ecf0f1; /* Light gray */
    text-decoration: none;
    margin-left: 20px;
    font-size: 16px;
    padding: 8px 15px;
    border-radius: 5px;
    transition: background-color 0.3s ease, color 0.3s ease;
  }

  .navbar a:hover {
    background-color: #3498db; /* Bright blue */
    color: #fff; /* White */
  }

  /* Container styling */
  .container {
    max-width: 1200px;
    margin: 20px auto;
    padding: 20px;
    background-color: #ffffff; /* White */
    border-radius: 10px;
    box-shadow: 0 4px 10px rgba(0, 0, 0, 0.1);
    min-height: 80vh; /* Ensure container takes up space */
  }

```

```

/* Responsive design */
@media (max-width: 768px) {
    .navbar {
        flex-direction: column;
        align-items: flex-start;
    }

    .navbar a {
        margin: 10px 0;
    }

    .container {
        margin: 10px;
        padding: 15px;
    }
}
</style>
</head>
<body>
    <div class="navbar">
        {% if user.is_authenticated %}
        <a href="{% url 'logout' %}">Logout</a>
        {% if user.is_staff %}
        <a href="{% url 'teacher_dashboard' %}">Dashboard</a> <!-- Updated URL
name -->
        {% else %}
        <a href="{% url 'student_dashboard' %}">Dashboard</a> <!-- Updated URL
name -->
        {% endif %}
        {% endif %}
    </div>
    <div class="container">
        {% block content %}{% endblock %}
    </div>
</body>
</html>

```

Description:

In a Django project, base.html serves as a **foundational template** that provides a consistent layout and structure across multiple pages. Here is a short note on its role in Django projects:

1. **Template Inheritance:** Acts as a **parent template** that other templates (e.g., login.html, student_dashboard.html) extend using `{% extends 'base.html' %}` to inherit its structure.
2. **Common Elements:** Contains shared HTML elements like the header, navbar, footer, and CSS/JS links, reducing code duplication.
3. **Block System:** Uses `{% block content %}` (and other blocks like title, styles) to define placeholders where child templates can insert specific content.
4. **Consistency:** Ensures a uniform look and feel (e.g., styling, navigation) across the site.
5. **Efficiency:** Simplifies maintenance by centralizing common design and functionality in one file.

login.html :

```
{% extends 'portal/base.html' %}
{% block content %}
<style>
  /* Header box styling */
  .header-box {
    text-align: center;
    padding: 20px;
    background-color: #3498db; /* Bright blue */
    color: #ffffff; /* White text */
    border-radius: 8px;
    margin-bottom: 30px;
    box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
  }

  .header-box h1 {
    margin: 0;
    font-size: 28px;
  }

  /* Login container styling */
  .login-container {
    max-width: 400px;
    margin: 0 auto;
    padding: 25px;
    background-color: #ecf0f1; /* Light gray */
    border-radius: 10px;
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.15);
  }

  .login-container h2 {
    text-align: center;
    color: #2c3e50; /* Dark blue */
    margin-bottom: 20px;
    font-size: 24px;
  }

  /* Message styling */
  .message {
    padding: 10px;
    margin-bottom: 15px;
    border-radius: 5px;
    text-align: center;
    font-size: 14px;
  }

  .message.success {
```



```

        background-color: #27ae60; /* Green */
        color: #ffffff;
    }

    .message.error {
        background-color: #c0392b; /* Red */
        color: #ffffff;
    }

    /* Form styling */
    form {
        display: flex;
        flex-direction: column;
        gap: 15px;
    }

    input[type="text"],
    input[type="password"] {
        padding: 12px;
        font-size: 16px;
        border: 1px solid #bdc3c7; /* Light gray border */
        border-radius: 5px;
        background-color: #ffffff; /* White */
        outline: none;
        transition: border-color 0.3s ease;
    }

    input[type="text"]:focus,
    input[type="password"]:focus {
        border-color: #3498db; /* Bright blue on focus */
    }

    input[type="submit"] {
        padding: 12px;
        font-size: 16px;
        background-color: #2c3e50; /* Dark blue */
        color: #ffffff; /* White text */
        border: none;
        border-radius: 5px;
        cursor: pointer;
        transition: background-color 0.3s ease;
    }

    input[type="submit"]:hover {
        background-color: #3498db; /* Bright blue on hover */
    }
</style>

<div class="header-box">
    <h1>Welcome to Assignment Submission System</h1>

```

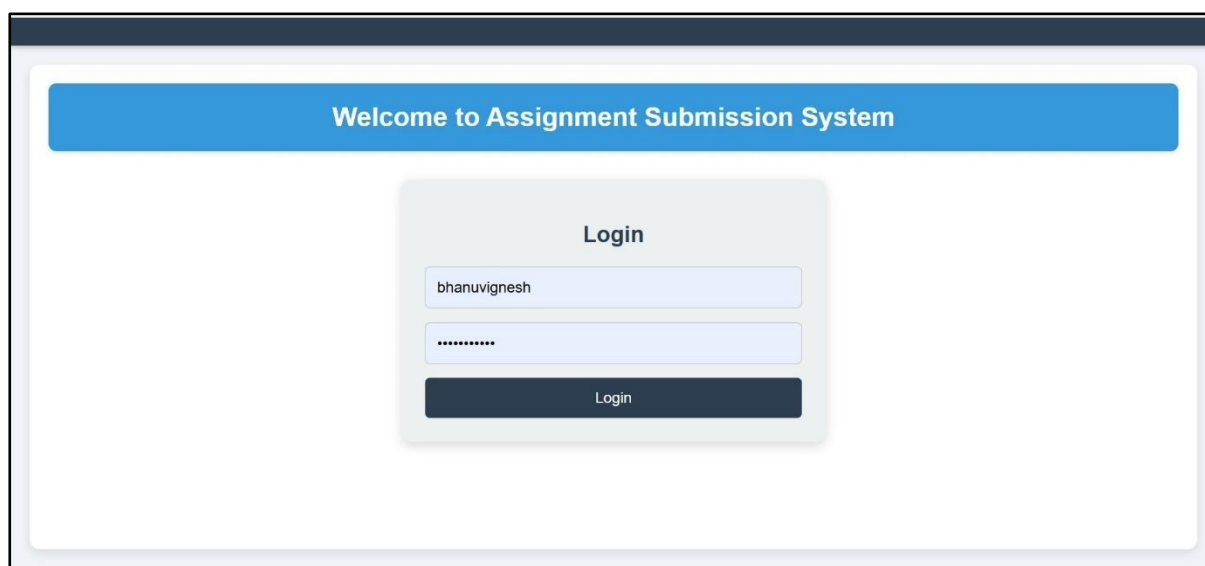
```
</div>
<div class="login-container">
  <h2>Login</h2>
  {% if messages %}
    {% for message in messages %}
      <div class="message {{ message.tags }}">{{ message }}</div>
    {% endfor %}
  {% endif %}
  <form method="post">
    {% csrf_token %}
    <input type="text" name="username" placeholder="Username" required>
    <input type="password" name="password" placeholder="Password" required>
    <input type="submit" value="Login">
  </form>
</div>
{% endblock %}
```

Description:

In a Django project, login.html is a template designed to handle **user authentication**. Here is a short note on its role in Django projects:

1. **User Authentication:** Provides a form for users to enter credentials (username and password) to log into the system.
2. **Form Submission:** Uses a POST method to send data to a login view for processing (e.g., `authenticate()` and `login()`).
3. **Extends Base:** Typically extends `base.html` to inherit common layout and styling.
4. **Feedback Display:** Shows messages (e.g., success or error) using Django's messages framework.
5. **Customization:** Can include styling and additional fields (e.g., "Remember Me") tailored to the project's needs.

OUTPUT:



The screenshot displays a web application interface for an assignment submission system. At the top, a blue banner reads "Welcome to Assignment Submission System". Below this, a light gray box contains a "Login" form. The form has two input fields: the first is labeled "bhanuvignesh" (the username) and the second is a password field with masked characters. A dark blue "Login" button is positioned below the password field.

Teacher_dashboard.html :

```
{% extends 'portal/base.html' %}
{% block content %}
<style>
/* General content styling */
h1 {
text-align: center;
color: #4a235a; /* Deep plum */
margin: 20px 0;
font-size: 32px;
}

h2 {
color: #6b3e7a; /* Muted purple */
margin: 30px 0 15px;
font-size: 24px;
}

/* Create new assignment link */
a[href="{% url 'create_assignment' %}"] {
display: inline-block;
padding: 10px 20px;
background-color: #2e7d32; /* Forest green */
color: #ffffff;
text-decoration: none;
border-radius: 5px;
margin: 10px 0;
transition: background-color 0.3s ease;
}

a[href="{% url 'create_assignment' %}"]:hover {
background-color: #4caf50; /* Lighter green */
}

/* Assignment card styling */
.assignment-card {
background-color: #f5f5dc; /* Light cream */
padding: 20px;
margin: 15px 0;
border-radius: 10px;
box-shadow: 0 3px 8px rgba(0, 0, 0, 0.1);
transition: transform 0.2s ease;
}

.assignment-card:hover {
transform: translateY(-5px);
}

.assignment-card h3 {
```

```

color: #4a235a; /* Deep plum */
margin: 0 0 10px;
font-size: 20px;
}

.assignment-card p {
margin: 5px 0;
color: #6b3e7a; /* Muted purple */
}

.assignment-card a {
color: #2e7d32; /* Forest green */
text-decoration: underline;
}

.assignment-card a:hover {
color: #4caf50; /* Lighter green */
}
</style>

<h1>Teacher Dashboard</h1>
<a href="{% url 'create_assignment' %}">Create New Assignment</a>
<h2>My Assignments</h2>
{% for assignment in assignments %}
<div class="assignment-card">
<h3>{{ assignment.title }}</h3>
<p>{{ assignment.description }}</p>
<p>Deadline: {{ assignment.deadline }}</p>
<a href="{% url 'view_submissions' assignment.id %}">View Submissions</a>
</div>
{% endfor %}
{% endblock %}

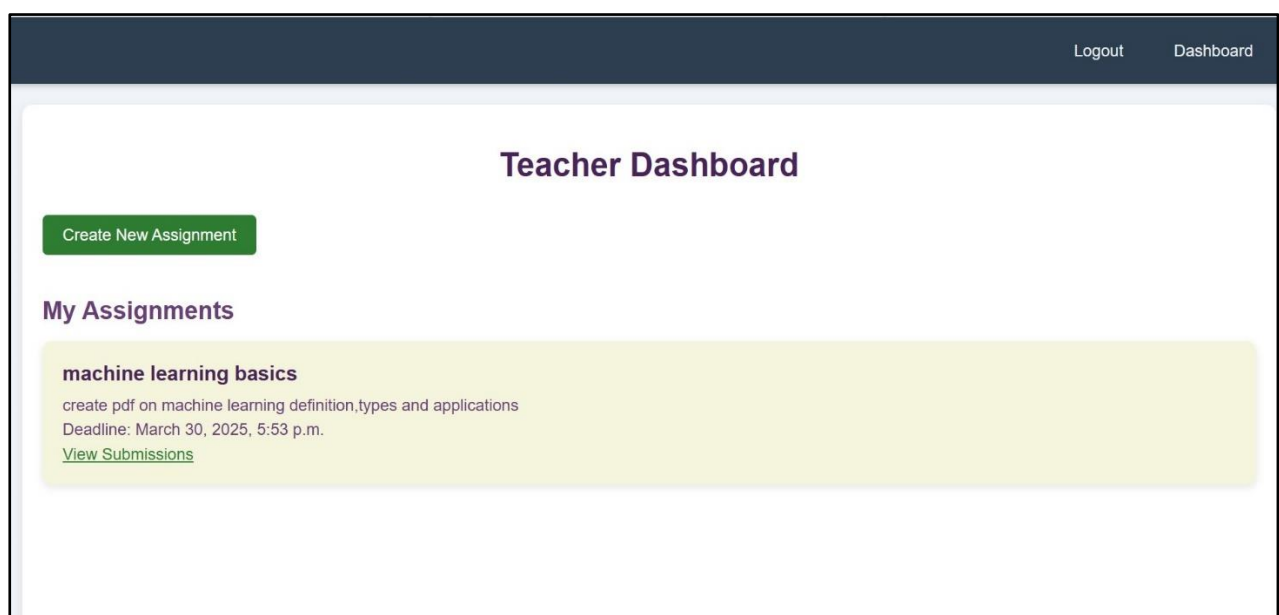
```

Description:

In an Assignment Submission System built with Django, `teacher_board.html` (Teacher Dashboard) serves as the **central interface** for teachers. Here's a short note on its role in Assignment submission system:

1. **Assignment Overview:** Displays a list of assignments created by the teacher, including titles, descriptions, and deadlines.
2. **Management Hub:** Provides links to create new assignments and view student submissions.
3. **Feedback Access:** Allows teachers to navigate to submission details for grading and feedback.
4. **Extends Base:** Inherits layout and styling from `base.html` for consistency.
5. **Role-Specific:** Tailored for teachers, restricting access to teacher-only features (e.g., via `is_staff` check).

OUTPUT:



student_dashboard.html :

```

{% extends 'portal/base.html' %}
{% block content %}
<style>
  /* General content styling */
  h1 {
    text-align: center;
    color: #3d2c5e; /* Deep purple */
    margin: 20px 0;
    font-size: 32px;
  }

  h2 {
    color: #5e2c3d; /* Dark reddish-purple */
    margin: 30px 0 15px;
    font-size: 24px;
  }

  /* Message styling */
  .message {
    padding: 10px;
    margin: 10px 0;
    border-radius: 5px;
    text-align: center;
    font-size: 14px;
  }

  .message.success {
    background-color: #27ae60; /* Green */
    color: #ffffff;
  }

  .message.error {
    background-color: #c0392b; /* Red */
    color: #ffffff;
  }

  /* Assignment card styling */
  .assignment-card {
    background-color: #fdf2e9; /* Light peach */
    padding: 20px;
    margin: 15px 0;
    border-radius: 10px;
    box-shadow: 0 3px 8px rgba(0, 0, 0, 0.1);
    transition: transform 0.2s ease;
  }

  .assignment-card:hover {
    transform: translateY(-5px);
  }

```

```

}

.assignment-card h3 {
    color: #3d2c5e; /* Deep purple */
    margin: 0 0 10px;
    font-size: 20px;
}

.assignment-card p {
    margin: 5px 0;
    color: #5e2c3d; /* Dark reddish-purple */
}

/* Submit button styling */
.submit-btn {
    display: inline-block;
    padding: 10px 20px;
    background-color: #e67e22; font-size: 16px;
    color: #ffffff;
    text-decoration: none;
    border-radius: 5px;
    background-color: #d35400; /* Vibrant orange */
    transition: background-color 0.3s ease;
}

.submit-btn:hover {
    background-color: #e67e22; /* Lighter orange */
}

/* Links in submissions */
a {
    color: #3d2c5e; /* Deep purple */
    text-decoration: underline;
}

a:hover {
    color: #5e2c3d; /* Dark reddish-purple */
}
</style>

<h1>Student Dashboard</h1>

{% if messages %}
    {% for message in messages %}
        <div class="message {{ message.tags }}">{{ message }}</div>
    {% endfor %}
{% endif %}

<h2>Available Assignments</h2>
{% for assignment in assignments %}

```



```

<div class="assignment-card">
  <h3>{{ assignment.title }}</h3>
  <p>{{ assignment.description }}</p>
  <p>Deadline: {{ assignment.deadline }}</p>
  {% if assignment.id in submitted_assignment_ids %}
    <p style="color: #4a2c6b">Status: Submitted</p>
  {% else %}
    {% if assignment.deadline > now %}
      <a href="{% url 'submit_assignment' assignment.id %}" class="submit-
btn">Submit Assignment</a>
    {% else %}
      <p style="color: #8b2c4a">Deadline Passed</p>
    {% endif %}
  {% endif %}
</div>
{% empty %}
  <p>No assignments available</p>
{% endfor %}

<h2>My Submissions</h2>
{% for submission in submissions %}
  <div class="assignment-card">
    <h3>{{ submission.assignment.title }}</h3>
    <p>Submitted on: {{ submission.submitted_at }}</p>
    <p>Grade: {{ submission.grade|default:'Not graded yet' }}</p>
    <p>Feedback: {{ submission.feedback|default:'No feedback yet' }}</p>
    <p>File: <a href="{{ submission.file.url }}"
target="_blank">Download</a></p>
  </div>
  {% empty %}
    <p>No submissions yet</p>
  {% endfor %}
{% endblock %}

```

Description:

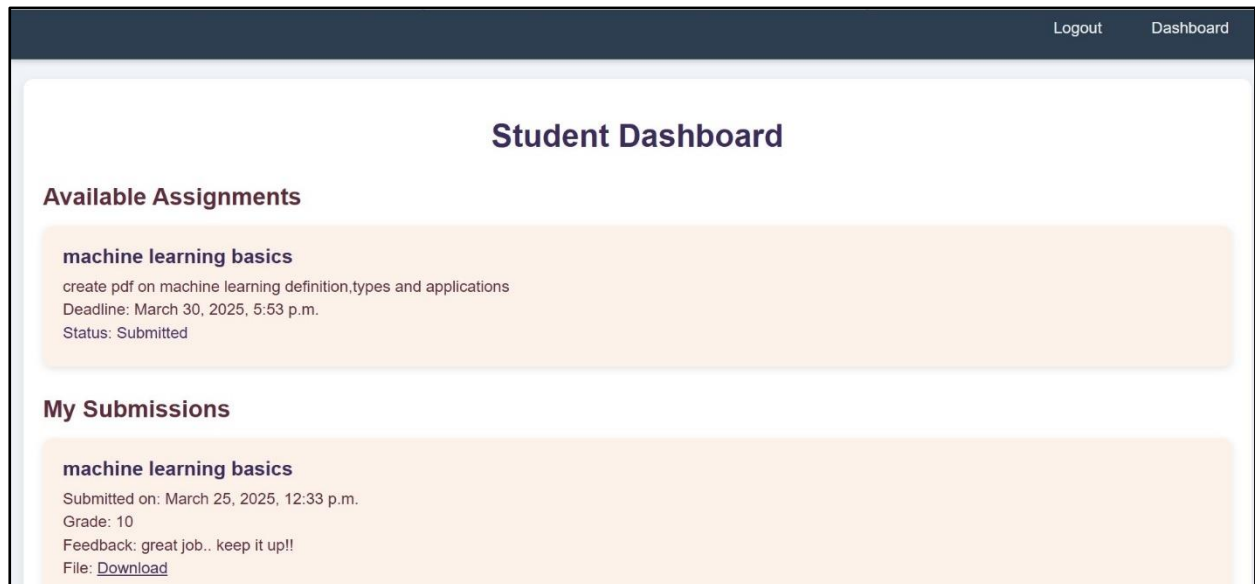
The Student Dashboard plays a critical role in empowering students to engage with assignments, submit work, and track their performance seamlessly within the system.

1. **Assignment Access:** Displays a list of available assignments with titles, descriptions, and deadlines for students to view.
2. **Submission Hub:** Provides options to submit assignments via links or buttons (e.g., "Submit Assignment") if deadlines are active.
3. **Progress Tracking:** Shows submission status (e.g., "Submitted" or "Deadline Passed") and personal submission history.
4. **Feedback View:** Allows students to see grades and feedback from teachers on submitted assignments.
5. **Extends Base:** Inherits layout and styling from base.html for a consistent look across the site.
6. **Role-Specific:** Tailored for students, limiting access to student-only features and data.

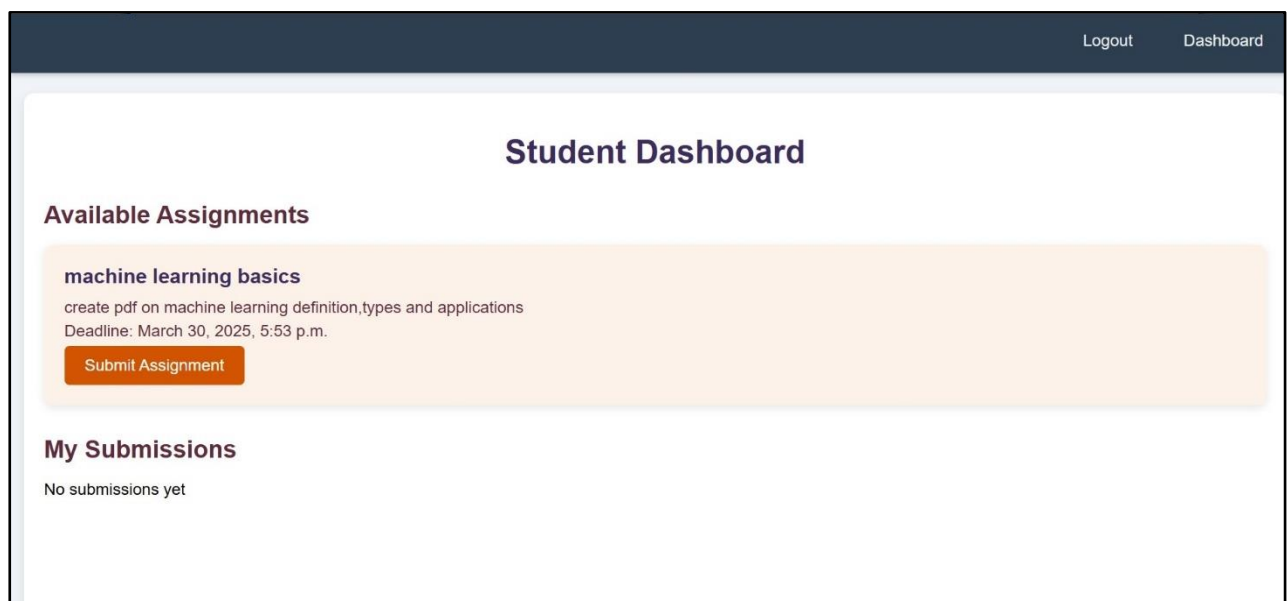
It serves as the student's main interface to interact with assignments, ensuring they can submit work and monitor their academic progress efficiently.

OUTPUT:

Student_dashboard after submitting assignment:



Student_dashboard before submitting assignment:



Assignment create.html :

```
{% extends 'portal/base.html' %}
{% block content %}
<style>
/* General content styling */
h1 {
text-align: center;
color: #4a235a; /* Deep plum */
margin: 20px 0;
font-size: 32px;
}

/* Form styling */
form {
max-width: 600px;
margin: 20px auto;
padding: 25px;
background-color: #f5f5dc; /* Light cream */
border-radius: 10px;
box-shadow: 0 3px 8px rgba(0, 0, 0, 0.1);
}

form p {
margin: 15px 0;
color: #6b3e7a; /* Muted purple */
}

form input[type="text"],
form textarea {
width: 100%;
padding: 10px;
font-size: 16px;
border: 1px solid #2e7d32; /* Forest green */
border-radius: 5px;
background-color: #ffffff;
box-sizing: border-box;
}

form input[type="submit"] {
padding: 12px 25px;
font-size: 16px;
background-color: #2e7d32; /* Forest green */
color: #ffffff;
border: none;
border-radius: 5px;
cursor: pointer;
transition: background-color 0.3s ease;
}
```

```

form input[type="submit"]:hover {
background-color: #4caf50; /* Lighter green */
}
</style>

<h1>Create Assignment</h1>
<form method="post">
{% csrf_token %}
{{ form.as_p }}
<input type="submit" value="Create">
</form>
{% endblock %}

```

Description:

The Assignment Create page is essential for teachers to populate the system with tasks, driving the core functionality of the submission process.

1. **Assignment Creation:** Provides a form for teachers to create new assignments by entering details like title, description, and deadline.
2. **Teacher Tool:** Enables teachers to define tasks that students will see and submit work for, initiating the assignment workflow.
3. **Form Submission:** Uses a POST method to send data to a view (e.g., `assignment_create`) for saving to the database.
4. **Extends Base:** Inherits layout and styling from `base.html` for consistency across the teacher interface.
5. **Input Fields:** Typically includes fields from a Django form (e.g., `{{ form.as_p }}`) for structured data entry.
6. **Role-Specific:** Accessible only to teachers (e.g., via `is_staff` check), ensuring controlled assignment management.
7. **System Integration:** Links new assignments to the teacher's dashboard and makes them available on the student dashboard.

OUTPUT:**Before creation of assignment:**

Create Assignment

Title:

Description:

Deadline:

Create

After creation of assignment:

Create Assignment

Title:

Description:

collect and submit real life applications of artificial intelligence

Deadline:

Create

Assignment submit.html :

```
{% extends 'portal/base.html' %}
{% block content %}
<style>
  /* General content styling */
  h1 {
    text-align: center;
    color: #3d2c5e; /* Deep purple */
    margin: 20px 0;
    font-size: 32px;
  }

  /* Form styling */
  form {

    max-width: 500px;
    margin: 20px auto;
    padding: 25px;
    background-color: #fdf2e9; /* Light peach */
    border-radius: 10px;
    box-shadow: 0 3px 8px rgba(0, 0, 0, 0.1);
  }

  form p {
    margin: 15px 0;
    color: #5e2c3d; /* Dark reddish-purple */
  }

  form input[type="file"] {
    padding: 10px;
    font-size: 16px;
    border: 1px solid #d35400; /* Vibrant orange */
    border-radius: 5px;
    background-color: #ffffff;
    width: 100%;
  }

  form input[type="submit"] {
    padding: 12px 25px;
    font-size: 16px;
    color: #ffffff;
    background-color: #d35400; /* Vibrant orange */
    border: none;
    border-radius: 5px;
    cursor: pointer;
    transition: background-color 0.3s ease;
  }
}
```

```
form input[type="submit"]:hover {
    background-color: #e67e22; /* Lighter orange */
}
</style>

<h1>Submit Assignment: {{ assignment.title }}</h1>
<form method="post" enctype="multipart/form-data">
    {% csrf_token %}
    {{ form.as_p }}
    <input type="submit" value="Submit">
</form>
{% endblock %}
```


Description:

The Assignment Submit page is crucial for enabling students to deliver their work, forming a key step in the assignment lifecycle within the system.

1. **Submission Interface:** Provides a form for students to upload their assignment files for a specific assignment.
2. **File Upload:** Uses a POST method with `enctype="multipart/form-data"` to handle file submissions to a view (e.g., `assignment_submission`).
3. **Student Action:** Allows students to submit work before the deadline, linking their effort to an assignment ID.
4. **Extends Base:** Inherits layout and styling from `base.html` for a consistent student experience.
5. **Form Display:** Renders a Django form (e.g., `{{ form.as_p }}`) with a file input field and submit button.
6. **Role-Specific:** Accessible only to authenticated students, tied to their user account for tracking.
7. **Integration:** Connects to the student dashboard, updating submission status and history upon successful upload.

OUTPUT:

Before assignment submission:

Submit Assignment: machine learning basics

File:

Choose File

No file chosen

Submit

After assignment submission:

Student Dashboard

Assignment submitted successfully

Available Assignments

machine learning basics

create pdf on machine learning definition,types and applications

Deadline: March 30, 2025, 5:53 p.m.

Status: Submitted

My Submissions

machine learning basics

Submitted on: March 26, 2025, 4:38 a.m.

Grade: Not graded yet

Feedback: No feedback yet

File: [Download](#)

Submission view.html :

```
{% extends 'portal/base.html' %}
{% block content %}
<style>
  /* General content styling */
  h1 {
    text-align: center;
    color: #4a235a; /* Deep plum */
    margin: 20px 0;
    font-size: 32px;
  }

  /* Assignment card styling */
  .assignment-card {
    background-color: #f5f5dc; /* Light cream */
    padding: 20px;
    margin: 15px 0;
    border-radius: 10px;
    box-shadow: 0 3px 8px rgba(0, 0, 0, 0.1);
    transition: transform 0.2s ease;
  }

  .assignment-card:hover {
    transform: translateY(-5px);
  }

  .assignment-card p {
    margin: 5px 0;
    color: #6b3e7a; /* Muted purple */
  }

  .assignment-card a {
    color: #2e7d32; /* Forest green */
    text-decoration: underline;
  }

  .assignment-card a:hover {
    color: #4caf50; /* Lighter green */
  }

  /* Form styling */
  form {
    display: flex;
    flex-direction: column;
    gap: 10px;
    margin-top: 10px;
  }

  form input[type="number"],
```

```

form textarea {
    width: 100%;
    padding: 10px;
    font-size: 16px;
    border: 1px solid #2e7d32; /* Forest green */
    border-radius: 5px;
    background-color: #ffffff;
    box-sizing: border-box;
}

form textarea {
    height: 100px;
    resize: vertical;
}

form input[type="submit"] {
    padding: 12px 25px;
    font-size: 16px;
    background-color: #2e7d32; /* Forest green */
    color: #ffffff;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    transition: background-color 0.3s ease;
}

form input[type="submit"]:hover {
    background-color: #4caf50; /* Lighter green */
}
</style>

<h1>Submissions for: {{ assignment.title }}</h1>
{% for submission in submissions %}
    <div class="assignment-card">
        <p>Student: {{ submission.student.username }}</p>
        <p>File: <a href="{{ submission.file.url }}">{{ submission.file.name }}</a></p>
        <form method="post">
            {% csrf_token %}
            <input type="hidden" name="submission_id" value="{{ submission.id }}">
            <input type="number" name="grade" value="{{ submission.grade }}"
placeholder="Grade">
            <textarea name="feedback" placeholder="Feedback">{{ submission.feedback
}}</textarea>
            <input type="submit" value="Save Feedback">
        </form>
    </div>
{% endfor %}
{% endblock %}

```

Description:

The submissionview.html page is vital for teachers to assess and provide feedback on student work, completing the evaluation process in the system.

1. **Submission Review:** Displays a list of student submissions for a specific assignment, allowing teachers to evaluate them.
2. **Teacher Interface:** Provides details like student names, uploaded files, grades, and feedback for each submission.
3. **Feedback Form:** Includes a form for teachers to input or update grades and feedback, submitted via POST to a view (e.g., submission_view).
4. **File Access:** Offers links to download submitted files (e.g., {{ submission.file.url }}) for review.
5. **Extends Base:** Inherits layout and styling from base.html for consistency in the teacher dashboard flow.
6. **Role-Specific:** Restricted to teachers (e.g., via is_staff check), ensuring only authorized users can grade submissions.
7. **Assignment Context:** Shows submissions tied to a specific assignment, identified by its title or ID.
8. **System Update:** Saves grading data to the database, reflecting changes in the student's submission history.

OUTPUT:

Before submission of feedback/view:

Student: sriram
File: [submissions/ml_assignment_ZQvUuCo.png](#)

Save Feedback

After submission of feedback/view:

Submissions for: machine learning basics

Student: bhanuvignesh
File: [submissions/ml_assignment.png](#)

Save Feedback

Student: sriram
File: [submissions/ml_assignment_ZQvUuCo.png](#)



DEPARTMENT OF INFORMATION TECHNOLOGY
JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM

Dr.Ch. Bindu Madhuri
 Asst. Professor & HOD

Email: hod. it@intugvcev.edu.in

1. Name of the Laboratory : Django Framework
 2. Name of the Student : R.Bhanu Vignesh
 3. Roll No : 23VV1A1244
 4. Class : II B-Tech II Semester
 5. Academic Year : 2024-25
 6. Name of Experiment : Database Integration and Configuration SQLITE
 7. Date of Experiment : 17-02-2025
 8. Date of Submission of Report : 21-02-2025

S,NO	ABILITY AND ACTIVITY	WEIGHTAGE OF MARKS	DAY TO DAY EVALUTION SCORE
1	Aim Objective, Tools required	3	
2	Theory, Algorithm and Observations	3	
3	Implementation	3	
4	Schematic diagrams, Architecture, workflow, Flowchart	3	
5	Tidiness of his/her working area, proper maintenance of system during and after experiment.	3	
	Total Score	15	

DATE:

Signature of Faculty :

Database Integration and Configuration – SQL LITE

1. **Migration Files:** Created by make migrations to record model changes (e.g., new fields, tables).
2. **Database Sync:** migrate executes these files to build or alter tables in the database.
3. **Automation:** Handles table creation, field types, and relationships without manual SQL.
4. **Order Matters:** Applies migrations sequentially to maintain database consistency.
5. **Default DB:** Works with the configured database (e.g., SQLite, PostgreSQL) in settings.py.

Step 1: Check if SQLite3 is Already Installed

sqlite3 --version

Step 2: Install SQLite3 (if not already installed)

On Windows:

1. Download the SQLite3 command-line tool from the official website:
SQLite Downloads
2. Download the "**Precompiled Binaries for Windows**" (usually a ZIP file).
3. Extract the ZIP file and place sqlite3.exe in a folder (e.g., C:\sqlite).
4. Add the folder to your system's **PATH** environment variable:
 - a. Search for "Environment Variables" in the Start menu.
 - b. Click on "Environment Variables."
 - c. In "System variables," select "Path" and click "Edit."
 - d. Add the folder path (e.g., C:\sqlite) and click OK.

On macOS:

sqlite3 --version

Step 3: Using SQLite3 with Django

Since Django uses SQLite3 as the default database, you don't need to install any additional drivers. Just make sure your settings.py file has the following configuration:


```
# Database
# https://docs.djangoproject.com/en/5.1/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

Step 4: Run Migrations to Create the Database

After setting up, run:

```
python manage.py migrate
```

Interacting directly with your Django project's SQLite database using the SQLite shell:

Step 1: Open the Django DB Shell

Make sure your virtual environment is activated, then use the following command:

```
python manage.py db shell
```

Step 2: Common SQLite Commands

Once you're inside the SQLite shell, you can use the following commands:

List All Tables:

```
.tables
```

Step3: View Table Schema:

```
.schema table_name;
```

Step4: Show All Data in a Table:

```
SELECT * FROM table_name;
```

Exit the SQLite Shell:

```
.exit
```



**DEPARTMENT OF INFORMATION TECHNOLOGY
JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM**

Dr.Ch. Bindu Madhuri
Asst. Professor & HOD

Email: hod. it@intugvcev.edu.in

1. Name of the Laboratory : Django Framework
2. Name of the Student : R.Bhanu Vignesh
3. Roll No : 23VV1A1244
4. Class : II B-Tech II Semester
5. Academic Year : 2024-25
6. Name of Experiment : Handling Forms in Django
7. Date of Experiment : 21-02-2025
8. Date of Submission of Report : 21-02-2025

S,NO	ABILITY AND ACTIVITY	WEIGHTAGE OF MARKS	DAY TO DAY EVALUTION SCORE
1	Aim Objective, Tools required	3	
2	Theory, Algorithm and Observations	3	
3	Implementation	3	
4	Schematic diagrams, Architecture, workflow, Flowchart	3	
5	Tidiness of his/her working area, proper maintenance of system during and after experiment.	3	
	Total Score	15	

DATE:

Signature of Faculty :

Handling Forms in Django

What is forms.py in Django:

In Django, forms.py is a Python file used to define custom forms for handling user input in a structured and secure way. It simplifies form creation, validation, and rendering, integrating seamlessly with Django's views and templates.

The forms.py file is crucial for managing user interactions, ensuring data integrity, and streamlining form handling in Django applications.

1. **Form Definition:** Contains classes (e.g., `class LoginForm(forms.Form)`) to specify fields like text, password, or file inputs.
2. **Validation:** Automatically validates input data (e.g., required fields, data types) and provides error messages.
3. **Rendering:** Generates HTML form elements when used in templates (e.g., `{{ form.as_p }}`) with customizable styling.
4. **Model Integration:** Can be tied to models using `ModelForm` to create/update database records (e.g., `AssignmentForm(forms.ModelForm)`).
5. **Security:** Includes CSRF protection by default and sanitizes input to prevent attacks like XSS.
6. **Flexibility:** Supports custom fields, widgets, and validation logic for specific project needs.
7. **Usage:** Typically located in an app directory (e.g., `myapp/forms.py`) and imported into views for processing.

Why Use forms.py:

- i. Simplifies creating and managing forms with less code.
- ii. Validates user input automatically for accuracy and safety.
- iii. Protects against security threats like CSRF attacks.
- iv. Reuses form logic across different parts of the app.
- v. Connects forms to database models for easy data handling.
- vi. Customizes fields and validation to fit specific needs.
- vii. Renders consistent, styled forms in templates effortlessly.

Types of Forms in Django:

Regular Forms (`forms.Form`):

- i. Basic form type for custom input fields (e.g., text, password) not tied to a model.
- ii. Example: A login form with username and password.

Model Forms (`forms.ModelForm`):

- i. Automatically generates forms from a model's fields for database operations (e.g., create, update).
- ii. Example: A form to create an Assignment with title and deadline.

Inline Forms (`inlineformset_factory`):

- i. Manages multiple instances of a model form in one page, often for related objects.
- ii. Example: Editing multiple Submission entries for an assignment.

Formsets (`formset_factory`):

- i. Handles multiple independent forms of the same type on a single page.
- ii. Example: A set of forms to collect feedback for multiple students.

Widget-Based Forms:

- i. Customizes form fields with widgets (e.g., Textarea, Select) for specific rendering.
- ii. Example: A description field as a large text area

forms.py:

```
from django import forms
from .models import Assignment, Submission

class AssignmentForm(forms.ModelForm):
    class Meta:
        model = Assignment
        fields = ['title', 'description', 'deadline']
        widgets = {
            'deadline': forms.DateTimeInput(attrs={'type': 'datetime-local'}),
        }

class SubmissionForm(forms.ModelForm):
    class Meta:
        model = Submission
        fields = ['file']
```



**DEPARTMENT OF INFORMATION TECHNOLOGY
JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM**

Dr.Ch. Bindu Madhuri
Asst. Professor & HOD

Email: hod. it@intugvcev.edu.in

1. Name of the Laboratory : Django Framework
2. Name of the Student : R.Bhanu Vignesh
3. Roll No : 23VV1A1244
4. Class : II B-Tech II Semester
5. Academic Year : 2024-25
6. Name of Experiment : Defining and Using Models
7. Date of Experiment : 21-02-2025
8. Date of Submission of Report : 07-03-2025

S,NO	ABILITY AND ACTIVITY	WEIGHTAGE OF MARKS	DAY TO DAY EVALUTION SCORE
1	Aim Objective, Tools required	3	
2	Theory, Algorithm and Observations	3	
3	Implementation	3	
4	Schematic diagrams, Architecture, workflow, Flowchart	3	
5	Tidiness of his/her working area, proper maintenance of system during and after experiment.	3	
	Total Score	15	

DATE:

Signature of Faculty :

Defining and Using Models

What is models.py in Django:

The models.py file is essential for structuring and interacting with the database in a Django project.

1. **Database Blueprint:** Defines the structure of database tables as Python classes (models) for data storage.
2. **ORM Mapping:** Uses Django's Object-Relational Mapping (ORM) to link Python code to database tables.
3. **Field Definitions:** Specifies fields (e.g., CharField, DateTimeField) and their properties (e.g., max_length, null).
4. **Relationships:** Establishes connections between models (e.g., ForeignKey, ManyToManyField) for relational data.
5. **Data Management:** Enables creating, updating, and querying database records via Python methods.
6. **Migration Source:** Generates migrations (via makemigrations) to apply model changes to the database.
7. **App-Specific:** Located in an app directory (e.g., myapp/models.py) to organize project data logic.

Why Use Django Models:

Django models streamline development, enhance maintainability, and abstract database complexity effectively.

Django models, defined in `models.py`, are a cornerstone of Django projects, enabling developers to manage databases efficiently using Python classes. They simplify data handling by providing an **ORM** (Object-Relational Mapping) layer, which eliminates the need for raw SQL and ensures seamless interaction with the database.

By defining fields and relationships, models enforce structure and validation, while automatic migrations keep the database schema in sync with the code. This abstraction not only boosts productivity but also makes applications portable across different database systems, making Django models indispensable for scalable and maintainable web development.

How to Apply Models:

- a. Define a model class with fields in `models.py`.
- b. Add the app to `INSTALLED_APPS` in `settings.py`.
- c. Run `python manage.py makemigrations` to create migration files.
- d. Use `python manage.py migrate` to apply changes to the database.
- e. Query or modify data in views using model methods like `.objects.all()`
- f. Create a `ModelForm` in `forms.py` to link the model to forms.
- g. Pass model data to templates for display via view context.

Models.py:

```
from django.db import models
from django.contrib.auth.models import User

class Assignment(models.Model):
    title = models.CharField(max_length=200)
    description = models.TextField()
    teacher = models.ForeignKey(User, on_delete=models.CASCADE)
    deadline = models.DateTimeField()
    created_at = models.DateTimeField(auto_now_add=True)

class Submission(models.Model):
    assignment = models.ForeignKey(Assignment, on_delete=models.CASCADE)
    student = models.ForeignKey(User, on_delete=models.CASCADE)
    file = models.FileField(upload_to='submissions/')
    submitted_at = models.DateTimeField(auto_now_add=True)
    grade = models.IntegerField(null=True, blank=True)
    feedback = models.TextField(null=True, blank=True)
```



**DEPARTMENT OF INFORMATION TECHNOLOGY
JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM**

Dr.Ch. Bindu Madhuri
Asst. Professor & HOD

Email: hod. it@intugvcev.edu.in

1. Name of the Laboratory : Django Framework
2. Name of the Student : R.Bhanu Vignesh
3. Roll No : 23VV1A1244
4. Class : II B-Tech II Semester
5. Academic Year : 2024-25
6. Name of Experiment : Migration-synchronizing models with database
7. Date of Experiment : 07-03-2025
8. Date of Submission of Report : 27-03-2025

S,NO	ABILITY AND ACTIVITY	WEIGHTAGE OF MARKS	DAY TO DAY EVALUATION SCORE
1	Aim Objective, Tools required	3	
2	Theory, Algorithm and Observations	3	
3	Implementation	3	
4	Schematic diagrams, Architecture, workflow, Flowchart	3	
5	Tidiness of his/her working area, proper maintenance of system during and after experiment.	3	
	Total Score	15	

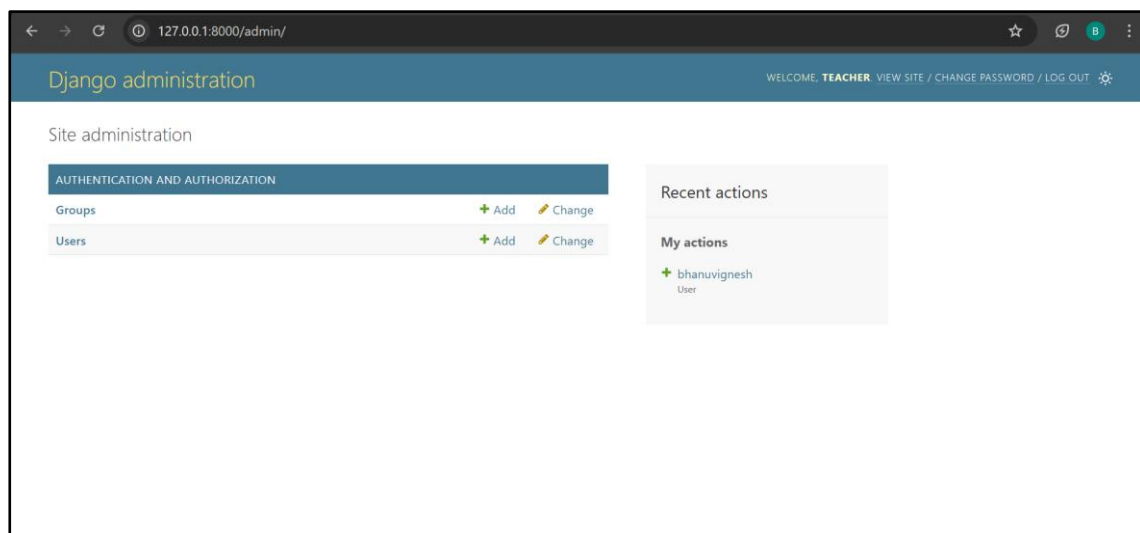
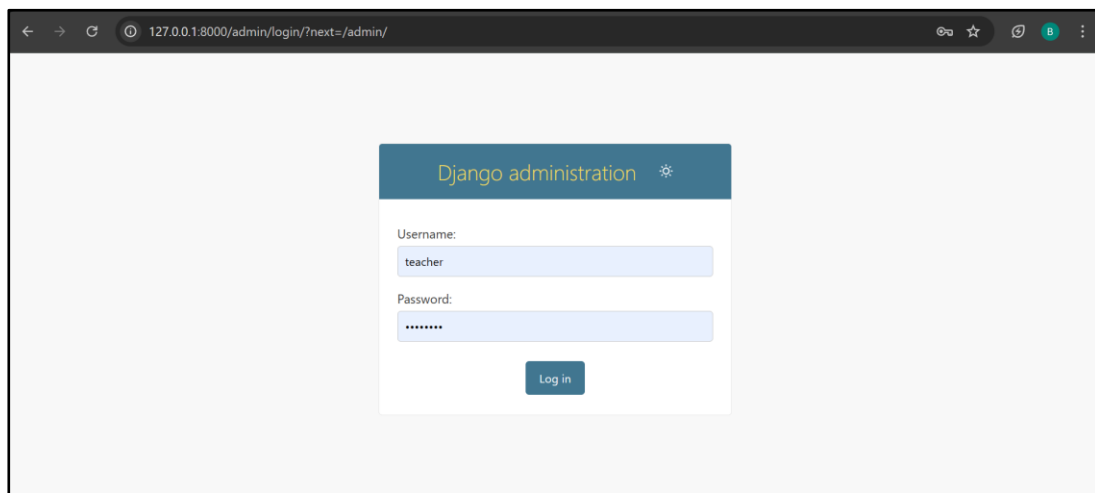
DATE:

Signature of Faculty :

Migrations: Synchronizing Models with the Database

How Databases store in Django:

1. Django models define how data is structured.
2. Migrations (python manage.py migrate) create tables in the database.



Run Migrations to Create Database Tables:

After defining your models, run the following commands to apply them to the database:

1. python manage.py makemigrations
2. python manage.py migrate



DEPARTMENT OF INFORMATION TECHNOLOGY
JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM

Dr.Ch. Bindu Madhuri
 Asst. Professor & HOD

Email: hod. it@intugvcev.edu.in

1. Name of the Laboratory : Django Framework
 2. Name of the Student : R.Bhanu Vignesh
 3. Roll No : 23VV1A1244
 4. Class : II B-Tech II Semester
 5. Academic Year : 2024-25
 6. Name of Experiment : Deploying Django application on cloud platforms
 7. Date of Experiment : 27-03-2025
 8. Date of Submission of Report : 04-04-2025

S,NO	ABILITY AND ACTIVITY	WEIGHTAGE OF MARKS	DAY TO DAY EVALUTION SCORE
1	Aim Objective, Tools required	3	
2	Theory, Algorithm and Observations	3	
3	Implementation	3	
4	Schematic diagrams, Architecture, workflow, Flowchart	3	
5	Tidiness of his/her working area, proper maintenance of system during and after experiment.	3	
	Total Score	15	

DATE:

Signature of Faculty :

Deploying Django Application on Cloud Platforms

What is Deployment?

Deployment is the process of making a Django web application live on the internet so users can access it. This involves hosting your app on a cloud server like AWS, Google Cloud, DigitalOcean, Heroku, or PythonAnywhere.

Features:

Scalability – Handle more users without performance issues.

Security – Protect user data with SSL and secure databases.

Global Accessibility – Users can access your app from anywhere.

Continuous Deployment – Easily update your app with new features.

Here's a step-by-step guide to **Register on GitHub**, **Create a Django website with login and registration pages**, and **Configure Django to handle static files**.

Step 1: Register on GitHub

1. Go to [GitHub](https://github.com) and click **Sign up**.
2. Enter your **Username, Email, and Password**.
3. Complete the verification and click **Create Account**.
4. Verify your email by clicking the link in your inbox.

Step 2: Push to GitHub

Initialize Git in your project: `git init`

2.Connect to GitHub: `git remote add origin`

`https://github.com/Tejaswiniy19/Assignment_Submission_Portal.git`

3.Add and commit changes: `git add .`

`git commit -m "Initial Commit: Login and Registration App"`

4.Push to GitHub: `git`

`branch -M main git`

`push -u origin main`

You have successfully built a **Django website** with login, registration, and static file management.

Your code is now available on **GitHub**.

GITHUB LINK: https://github.com/bhanuvignesh44/Assignment_Submission_Portal.git

Front End Web Developer Certificate

